

Python Based Lap Simulation

Adriel Torresola

05/21/2025

Abstract

As in all facets of engineering, having the ability to simulate distinct outcomes of varying real world parameters is crucial for tuning such parameters to achieve peak performance. Within motorsports this is especially important as there are dozens of parameters that can have real-world effects on the speed, efficiency, or even the measured loads at crucial points on the vehicle. This project seeks to bridge the gap within existing open-source lap simulation software that has been found to be missing crucial features for electric vehicles (such as torque modulation, motor usage control, and driver input optimization)

Summarized Methodology

In the existing software applications found, the two common solving methods are direct analytical solving (for simpler pieces of software such as OptimumLap) or much more complex full track optimization as what can be more common for paid lap simulators such as ChassisSim. The proposed method leans more towards a systems controls method of simulating the driver and vehicle around a track. This allows us to implement complex physical models for the vehicle without negatively impacting the simulation time as the solver does not have to find the absolute best line around track, rather it simulates a good driver.

The lap calculation methodology is as follows;

1. Define Parameters for:
 - car.py
 - tires.py
 - track.py
 - simulation.py
 - motor.py
2. Once the simulation has started, the program will cycle through all of the segments of track, thus calling a sub-module within simulation.py to find the right set of vehicle controls to iterate through the next few segments of track.
3. If the next 5 segments are able to be achieved, the first segment's inputs are applied to the final model.
 - Achieving a segment input implies that the set of throttle, brake, and steering inputs are enough to get the vehicle to drive on the given racing line (with a % lateral tolerance).
4. Once the segment has been verified, the inputs can be added to the final model and the next segment can start to be calculated.

- car.py
 - Handles most of the highest level logic
 - Most of the parts of the simulation in terms of things that the car does are handled by this part of the script.
 - References most of the other scripts
 - Generates for example the relevant tire model itself
- tires.py
 - One of the most important single parts of the car
 - Inherently handles all of the tire-specific calculations
 - Divided into physical tire and magic formula tire:
- motor.py
 - Handles the actual input/output of determining if a given torque can be provided
 - Uses the motor power curves to determine whether a specific torque (given RPM) is valid, and then determines what the operating characteristics will thus be.
 - Will have functions to limit torque based on the transient heat parameters
- track.py
 - Class to create and use tracks
 - Has visualization methods
- simulation.py
 - Combines all of the previous scripts to get the movement of the car around the track.
 - Maybe consider dividing into different simulation types?

Simulation

Physical Model

Car

1. 5 vector model
 - Four Tires
 - Center of Gravity
2. As throttle is applied, slip ratio is increased (thus accelerating car)
3. As steering angle is applied, slip angle is introduced which causes lateral acceleration.
4. Any time that inputs are applied to the model, all forces are calculated subsequently based on the applied slip ratio and slip angle.

— **Summary:**

Physical model works by applying throttle and steering to a given situation and it thus simulates the resultant movement of the vehicle.

Process Overview

Necessary Functions

Time step function

- Timestep
 - inputs:
 - Steering angle (must be within +- of previous) (basically ensure plausible steering velocity)
 - Throttle input (0-100%) (only varies the provided torque) (torque can be 0-100% of available torque at that given rpm)
 - outputs:
 - Calculates the torque based on previous
 - Calculates the slip angle of all the tires
 - Calculates the slip ratio of all the tires
 - From the slip angle and slip ratio, calculates the resultant forces of the tires
 - If the resultant forces exceed the capacity of the tires, we modify either of the inputs until it does work.

Motor Function

- Parts
 - inputs
 - throttle %
 - rpm
 - outputs
 - $\text{output torque} = \text{throttle \%} \times \text{max available torque @ RPM}$
 - above continuous torque (flag)
 - overheat_value (rms of power usage)
 - parameters
 - continuous torque curve
 - max torque curve

Steering Function

- Parts
 - inputs
 - steering angle %
 - outputs
 - sets the steering angle of the steered wheels
 - does not apply the steering automatically, just provides the correct values
 - verifies that the steering angle “speed” is not excessive (checks that the delta angle over time does not exceed an amount)
 - parameters
 - ratio between steering angle and actual steering performed per wheel

Physical Tire

tire_class

Holds all non-magic formula tire parameters and methods divided into the following categories:

Static parameters

- width
- diameter
- rotational inertia

Transient parameters

- steering angle
- angular velocity
- angular acceleration
- slip angle?

Methods

Magic Formula Tire

new_magic_formula_tire

Holds all the parameters and methods to do with calculating tire forces themselves.

Static parameters

- Magic tire formula parameters

Transient parameters

- longitudinal tire forces
- lateral tire forces